

Visitor, Chain of Responsibility e Mediator

Padrões de Projeto - Aula #10

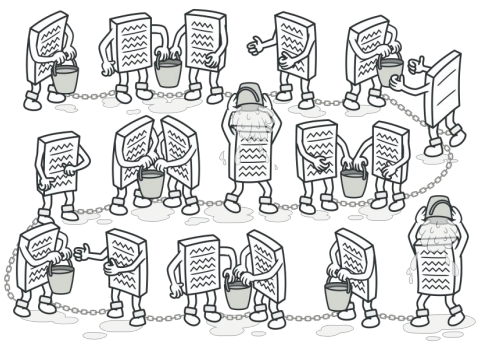
Prof. Dr. Carlos Melo

carlos.melo@upe.br

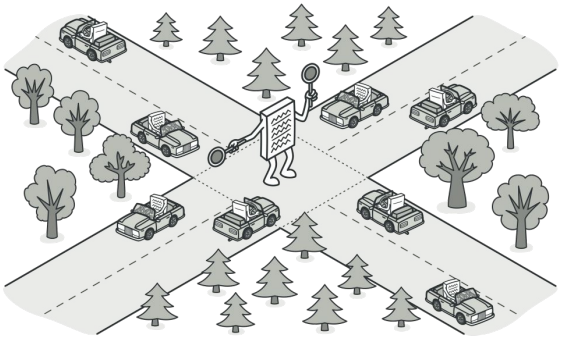
UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Padrões Comportamentais (parte #3)

Chain of Responsibility
Encadeia objetos que podem tratar um pedido e decidir se ele continua pela corrente



Mediator
Centraliza as regras de comunicação entre componentes em um objeto mediador



Visitor
Separar operações dos objetos sobre os quais elas são executadas



- No primeiro, o foco está no encaminhamento do pedido
- No segundo, está na coordenação dos componentes
- Já o terceiro foca na gestão das ações por objetos de diferentes tipos

carlos.melo@upe.br

Objetivos de aprendizagem

- Implementar *handlers* com uma interface comum e uma referência para o próximo elo
- Montar correntes e acompanhar quando um pedido continua ou tem seu processamento interrompido
- Transferir as regras de colaboração entre componentes para um mediador
- Implementar visitantes que realizam operações diferentes sobre os mesmos objetos
- Identificar quando utilizar cada padrão e avaliar seus custos

Visitor - O Problema

- Uma aplicação trabalha com cidades, indústrias e pontos turísticos
- Precisamos exportar suas informações para `XML` e, posteriormente, para outros formatos
  - `Cidade` exporta nome e população
  - `Industria` exporta nome e setor
  - `PontoTuristico` exporta nome e categoria
- Adicionar métodos como `exportar_xml()` e `exportar_json()` em cada classe espalharia a lógica de exportação pelas classes do domínio

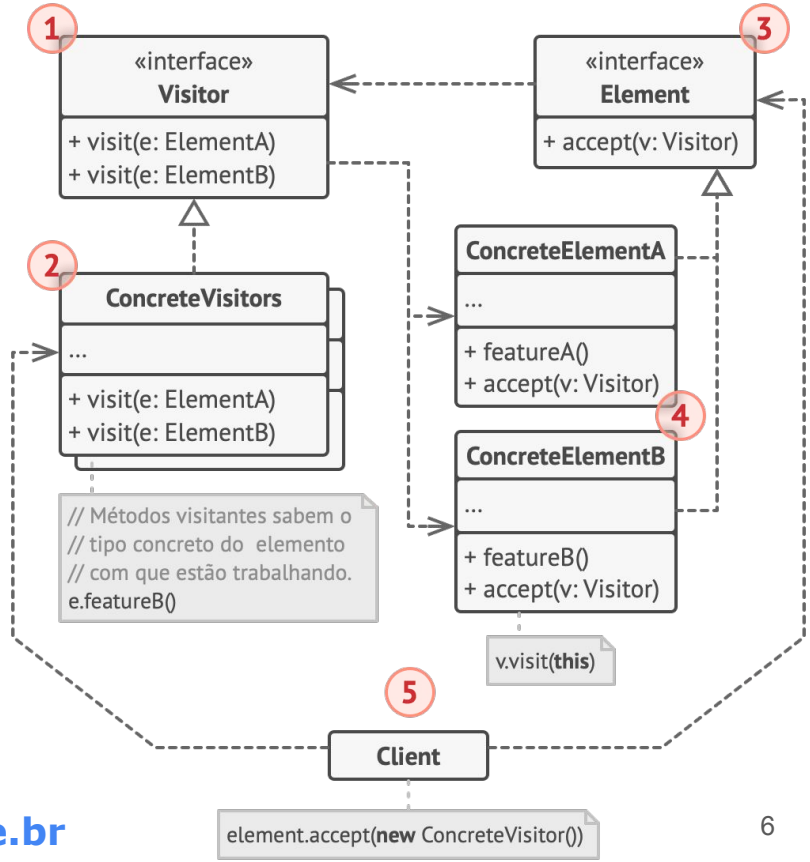
Visitor - ideia e intenção

"Separar os algoritmos dos objetos sobre os quais eles operam"

- Ideia central:
  - O Visitante reúne uma operação com tratamentos específicos para diferentes tipos de elementos
  - Cada Elemento aceita um visitante
  - O Elemento Concreto encaminha a chamada ao método correspondente ao seu tipo
- No nosso contexto `ExportadorXML` e `ExportadorJSON` processam os mesmos objetos, produzindo representações diferentes

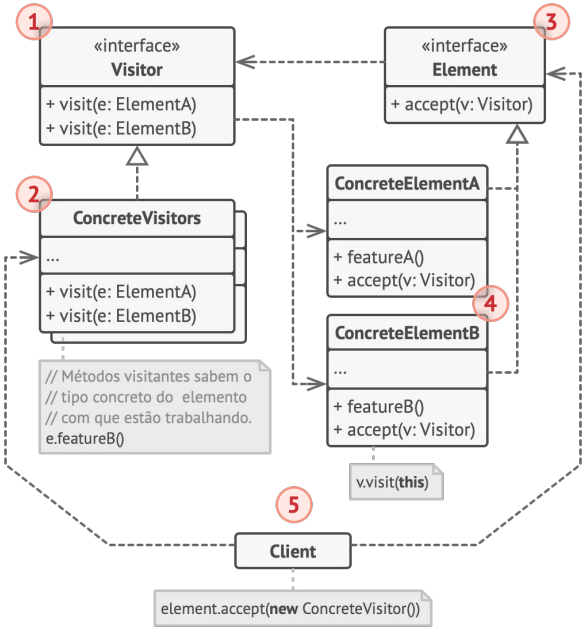
Visitor - Estrutura

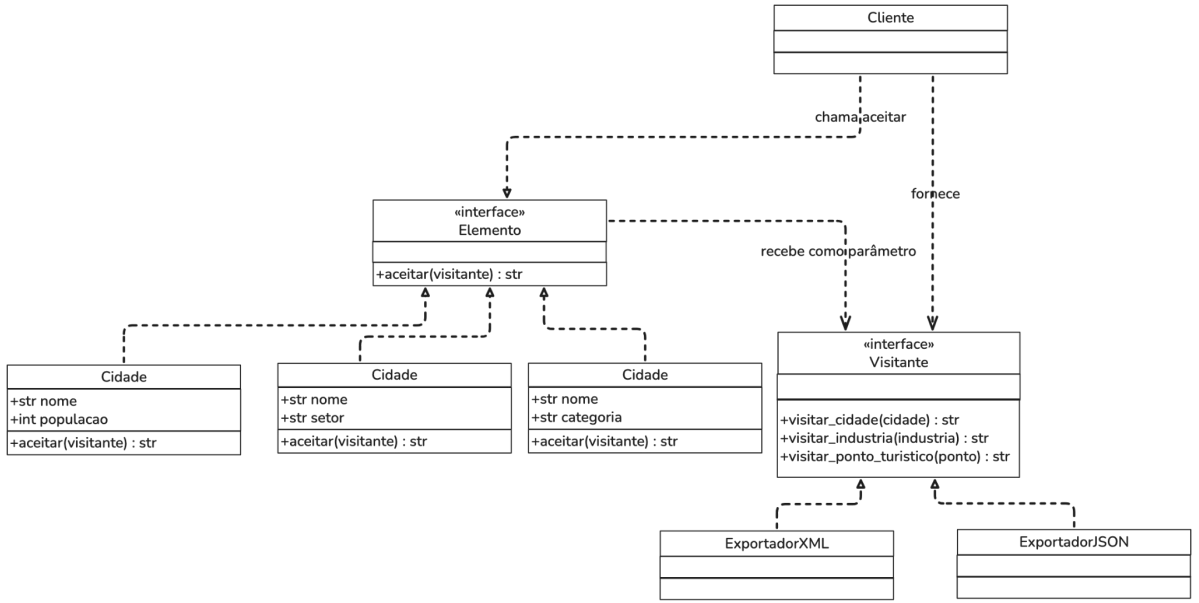
1. A interface `Visitor` declara um conjunto de métodos visitantes que podem receber elementos concretos de uma estrutura de objetos como argumentos
2. Cada `ConcreteVisitor` implementa diversas versões do mesmo comportamento, feitos sob medida para diferentes elementos concretos de classes
3. A interface `Element` declara um método `accept` para visitantes
4. Cada `ConcreteElement` deve implementar o método `accept` que redireciona a chamada para o método `visit` apropriado que corresponde com a atual classe elemento
5. O `Client` geralmente representa uma coleção de outros objetos complexos (e.g árvore Composite)



carlos.melo@upe.br

Aplicando ao nosso contexto





Aplicando ao nosso contexto - Interfaces

```python
from __future__ import annotations
from abc import ABC, abstractmethod


class Elemento(ABC):
    @abstractmethod
    def aceitar(self, visitante: Visitante) -> str:
        pass


class Visitante(ABC):
    @abstractmethod
    def visitar_cidade(self, cidade: Cidade) -> str:
        pass

    @abstractmethod
    def visitar_industria(self, industria: Industria) -> str:
        pass

    @abstractmethod
    def visitar_ponto_turistico(self, ponto: PontoTuristico) -> str:
        pass
```

- Elemento: exige o método de aceitação
- Visitante: declara um método para cada tipo de Elemento Concreto

Aplicando ao nosso contexto - Elementos concretos

```python
from dataclasses import dataclass


@dataclass
class Cidade(Elemento):
    nome: str
    populacao: int

    def aceitar(self, visitante: Visitante) -> str:
        return visitante.visitar_cidade(self)


@dataclass
class Industria(Elemento):
    nome: str
    setor: str

    def aceitar(self, visitante: Visitante) -> str:
        return visitante.visitar_industria(self)


@dataclass
class PontoTuristico(Elemento):
    nome: str
    categoria: str

    def aceitar(self, visitante: Visitante) -> str:
        return visitante.visitar_ponto_turistico(self)
```

- Cada classe encaminha a chamada ao método correspondente do visitante
- A lógica de exportação ficará nos Visitantes Concretos

Aplicando ao nosso contexto - ExportadorXML

```python
from xml.etree.ElementTree import Element, tostring


class ExportadorXML(Visitante):
    def visitar_cidade(self, cidade: Cidade) -> str:
        no = Element("cidade", nome=cidade.nome, populacao=str(cidade.populacao))
        return tostring(no, encoding="unicode")

    def visitar_industria(self, industria: Industria) -> str:
        no = Element("industria", nome=industria.nome, setor=industria.setor)
        return tostring(no, encoding="unicode")

    def visitar_ponto_turistico(self, ponto: PontoTuristico) -> str:
        no = Element("ponto_turistico", nome=ponto.nome, categoria=ponto.categoria)
        return tostring(no, encoding="unicode")
```

- Element monta o nó `XML`, e `tostring()` produz sua representação textual
- Cada método utiliza os campos específicos do elemento recebido

Aplicando ao nosso contexto - ExportadorJSON

```python
import json


class ExportadorJSON(Visitante):
    def visitar_cidade(self, cidade: Cidade) -> str:
        return json.dumps({
          "tipo": "cidade",
          "nome": cidade.nome,
          "populacao": cidade.populacao
        }, ensure_ascii=False)

    def visitar_industria(self, industria: Industria) -> str:
        return json.dumps({
          "tipo": "industria",
          "nome": industria.nome,
          "setor": industria.setor}, ensure_ascii=False
        )

    def visitar_ponto_turistico(self, ponto: PontoTuristico) -> str:
        return json.dumps({
          "tipo": "ponto_turistico",
          "nome": ponto.nome, "categoria": ponto.categoria}, ensure_ascii=False,
        )
```

- `json.dumps()` transforma o dicionário em texto JSON
- O novo formato foi implementado sem alterar `Cidade`, `Industria` ou `PontoTuristico`

Aplicando ao nosso contexto - Execução

```python
elementos: list[Elemento] = [
    Cidade("Vila Nova", 30000),
    Industria("Fábrica Alfa", "alimentos"),
    PontoTuristico("Mirante Central", "paisagem"),
]

for visitante in (ExportadorXML(), ExportadorJSON()):
    print(type(visitante).__name__)
    for elemento in elementos:
        print(elemento.aceitar(visitante))
```

Prós e Contras do Visitor

Prós

- Acrescenta operações por meio de novos Visitantes Concretos
- Reúne os tratamentos de uma operação em uma classe
- Pode acumular resultados durante as visitas

Contras

- Acrescentar Elementos Concretos exige atualizar os visitantes
- Os visitantes precisam conhecer os tipos concretos dos elementos
- Pode precisar de dados que os elementos mantêm encapsulados

Use

- Quando uma operação precisa tratar objetos de diferentes tipos
- Quando comportamentos auxiliares, como exportação, devem ficar separados das classes do domínio
- Quando novas operações precisam ser acrescentadas com frequência.

Chain of Responsibility - O Problema

- Temos um sistema de pedidos de uma loja virtual
  - Apenas usuários autenticados podem criar pedidos
  - Algumas operações exigem permissões administrativas
  - As verificações precisam ocorrer antes da operação solicitada
- Por exemplo:
  - O sistema recebe uma solicitação
  - Verifica a autenticação
  - Verifica a permissão necessária
  - Executa a operação, caso as verificações sejam aprovadas
- Se a autenticação falhar, as etapas seguintes devem ser interrompidas



carlos.melo@upe.br

Chain of Responsibility - O Problema

- Novos requisitos acrescentam outras verificações:
  - Validar os dados recebidos
  - Limitar tentativas repetidas
  - Consultar o cache em consultas que possam reaproveitar uma resposta
- Concentrar tudo em um único método dificulta:
  - Reutilizar apenas algumas verificações em outro fluxo
  - Alterar a ordem das etapas
  - Acrescentar uma verificação sem editar a sequência central
- Além disso, cada etapa pode produzir uma decisão diferente:
  - Se a verificação for aprovada então continua
  - Se a solicitação for inválida então interrompemos
  - Se a resposta for encontrada em *cache*, podemos retorna-la e encerrar o processamento
- Precisamos separar as etapas e controlar como o pedido passa entre elas

Chain of Responsibility - A ideia

- Separar cada etapa em um objeto chamado *handler*
  - Todos os *handlers* seguem uma interface comum
  - Cada *handler* mantém uma referência para o próximo elo
  - Ao receber uma solicitação, ele decide:
    - Se executa algum processamento
    - Se encaminha a solicitação ou encerra o fluxo
- O Cliente cria os *handlers* e configura suas ligações
- Na loja virtual, autenticação, permissão e validação passam a ser etapas independentes, combinadas em uma corrente

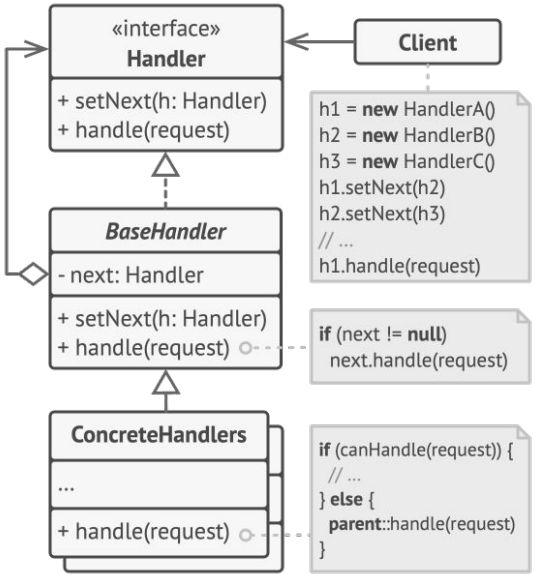
Chain of Responsibility - Intenção

"Evitar que o remetente dependa diretamente de todos os possíveis responsáveis pelo tratamento de uma solicitação"

- O padrão admite duas formas de organizar o processamento:
  - *Tratar e encerrar*: o primeiro *handler* capaz de atender à solicitação produz a resposta
  - *Processar e continuar*: um *handler* executa sua etapa e encaminha a solicitação, enquanto houver condições para continuar
- Na loja virtual, uma verificação aprovada permite continuar, enquanto que uma reprovação interrompe o fluxo.

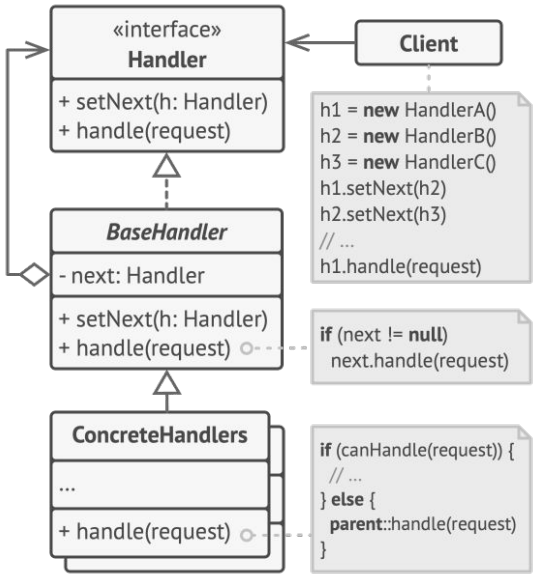
Chain of Responsibility - Estrutura

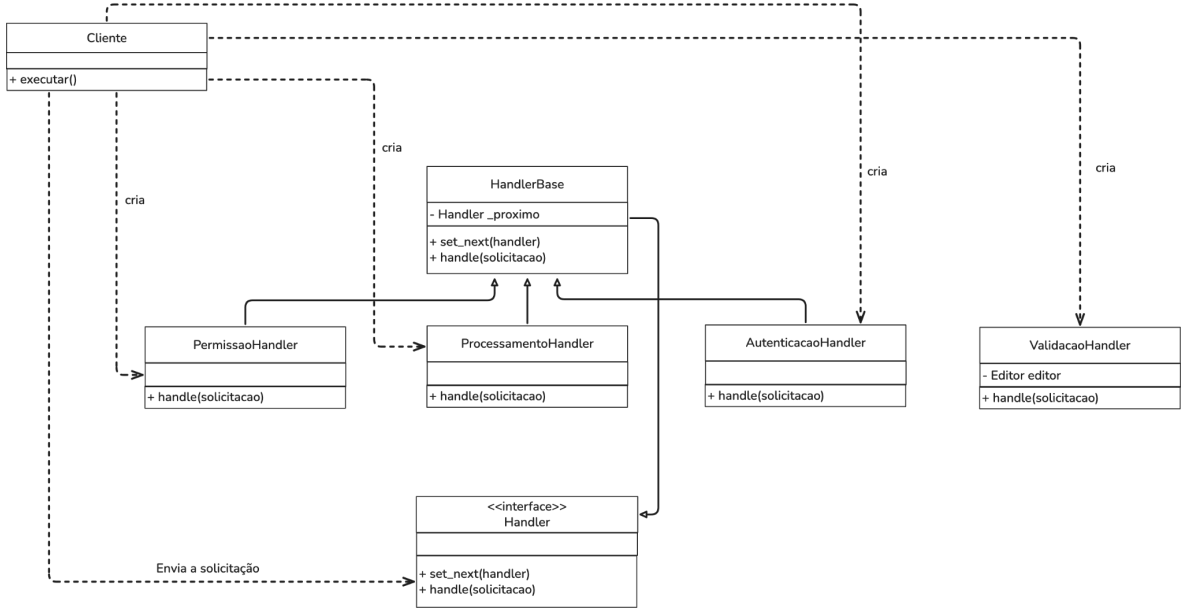
1. O **Handler** declara o contrato de tratamento e a configuração do próximo elo
2. O **Handler Base** concentra o encaminhamento padrão e é opcional
3. ConcreteHandlers implementam o processamento e a decisão de continuar
4. O Client monta a corrente e envia solicitações a um de seus handlers



carlos.melo@upe.br

Aplicando ao nosso contexto





Aplicando ao nosso contexto - Solicitação

```python
from dataclasses import dataclass


@dataclass
class Solicitacao:
    usuario: str | None
    perfil: str
    operacao: str
    quantidade: int = 0
```

```python
solicitacao = Solicitacao(
    usuario="ana",
    perfil="cliente",
    operacao="criar_pedido",
    quantidade=2,
)
```

- `usuario=None` representa uma solicitação sem usuário autenticado
- `listar_todos` exigirá o perfil "administrador"
- `criar_pedido` exigirá uma quantidade maior que zero
- As verificações serão responsabilidade dos *handlers*

Aplicando ao nosso contexto - HandlerInterface

```python
from abc import ABC, abstractmethod


class Handler(ABC):
    @abstractmethod
    def set_next(self, handler: "Handler") -> "Handler":
        pass

    @abstractmethod
    def handle(self, solicitacao: Solicitacao) -> str | None:
        pass
```

- `set_next()` configura o próximo handler
- `handle()` recebe a solicitação e pode produzir uma resposta
- `None` representa o encerramento da corrente sem uma resposta
- O Cliente pode enviar a solicitação usando esse contrato, independentemente da classe concreta do handler

Aplicando ao nosso contexto - HandlerBase

```python
class HandlerBase(Handler):
    def __init__(self) -> None:
        self._proximo: Handler | None = None

    def set_next(self, handler: Handler) -> Handler:
        self._proximo = handler
        return handler

    def handle(self, solicitacao: Solicitacao) -> str | None:
        if self._proximo is not None:
            return self._proximo.handle(solicitacao)

        return None
```

- `_proximo` mantém a referência para o próximo elo
- O comportamento padrão é encaminhar a solicitação
- Sem um próximo *handler*, retorna `None`
- `set_next()` retorna o *handler* recebido, permitindo configurar sucessivos elos por chamadas encadeadas

Aplicando ao nosso contexto - Autenticação e Permissão

```python
class AutenticacaoHandler(HandlerBase):
    def handle(self, solicitacao: Solicitacao) -> str | None:
        if solicitacao.usuario is None:
            return "Acesso negado"
        return super().handle(solicitacao)


class PermissaoHandler(HandlerBase):
    def handle(self, solicitacao: Solicitacao) -> str | None:
        if (
            solicitacao.operacao == "listar_todos"
            and solicitacao.perfil != "administrador"
        ):
            return "Acesso negado"
        return super().handle(solicitacao)
```

- Uma reprovação retorna uma resposta imediatamente
- Uma aprovação chama `super().handle()`, que encaminha a solicitação
- A etapa de permissão será alcançada somente se a autenticação permitir continuar

Aplicando ao nosso contexto - Validação e Processamento

```python
class ValidacaoHandler(HandlerBase):
    def handle(self, solicitacao: Solicitacao) -> str | None:
        if solicitacao.operacao not in {"criar_pedido", "listar_todos"}:
            return "Solicitação inválida: operação desconhecida."
        if (
            solicitacao.operacao == "criar_pedido"
            and solicitacao.quantidade <= 0
        ):
            return "Solicitação inválida: quantidade deve ser maior que zero."
        return super().handle(solicitacao)


class ProcessamentoHandler(HandlerBase):
    def handle(self, solicitacao: Solicitacao) -> str | None:
        if solicitacao.operacao == "criar_pedido":
            return (
                f"Pedido criado para {solicitacao.usuario}: "
                f"{solicitacao.quantidade} item(ns)."
            )
        if solicitacao.operacao == "listar_todos":
            return "Listagem de todos os pedidos autorizada."
        return None
```

- A validação aprovada encaminha a solicitação
- O processamento produz a resposta final

Aplicando ao nosso contexto - Cliente

```python
class Cliente:
    @staticmethod
    def executar() -> None:
        autenticacao = AutenticacaoHandler()
        permissao = PermissaoHandler()
        validacao = ValidacaoHandler()
        processamento = ProcessamentoHandler()
        autenticacao.set_next(permissao)
        permissao.set_next(validacao)
        validacao.set_next(processamento)
        solicitacao = Solicitacao(
            usuario="ana",
            perfil="cliente",
            operacao="criar_pedido",
            quantidade=2,
        )
        print(autenticacao.handle(solicitacao))
Cliente.executar()
```

- O Cliente monta a corrente e envia a solicitação ao primeiro *handler*
- Cada etapa controla o encaminhamento para a seguinte

Prós e contras do Chain of Responsibility

Prós

- Permite controlar a ordem de tratamento
- Facilita reutilizar handlers em diferentes correntes
- Permite acrescentar handlers sem alterar as classes dos existentes

Contras

- A ordem dos handlers influencia o resultado
- O fluxo pode exigir acompanhar várias chamadas entre objetos
- Uma corrente pode terminar sem que algum handler produza uma resposta
- No nosso contrato, isso resulta em `None`

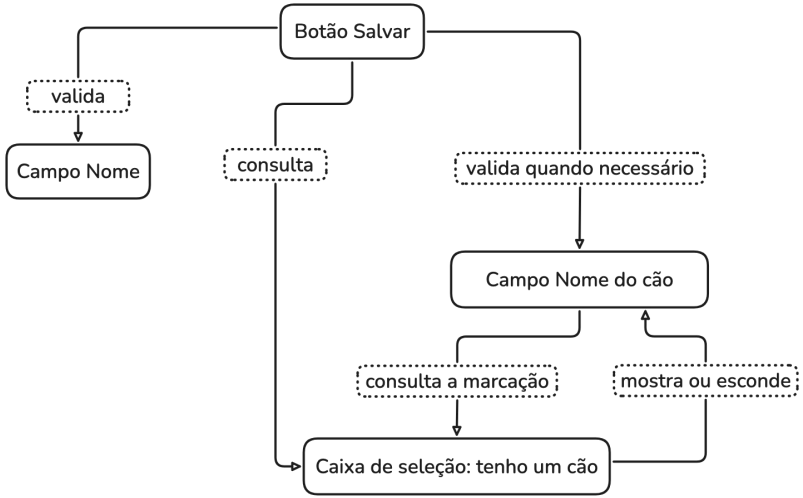
Use:

- Quando as etapas precisam ser combinadas, reutilizadas ou reorganizadas, com possibilidade de interromper o processamento

Mediator - O Problema

- Temos um formulário de edição de perfil que contém:
  - Campo de nome
  - Caixa de seleção “Tenho um cão”
  - Campo de nome do cão
  - Botão Salvar
- As regras conectam esses elementos
  - Duplicação da operação em diferentes elementos
  - Dependências entre menus, atalhos e botões
  - Alterações na interface gráfica sempre que a lógica da operação mudar

Mediator - Dependências entre componentes



- O botão precisa conhecer vários campos
- A caixa de seleção conhece o campo que controla
- Reutilizar esses componentes exige lidar com as dependências específicas do formulário
- As regras de colaboração ficam espalhadas entre os elementos

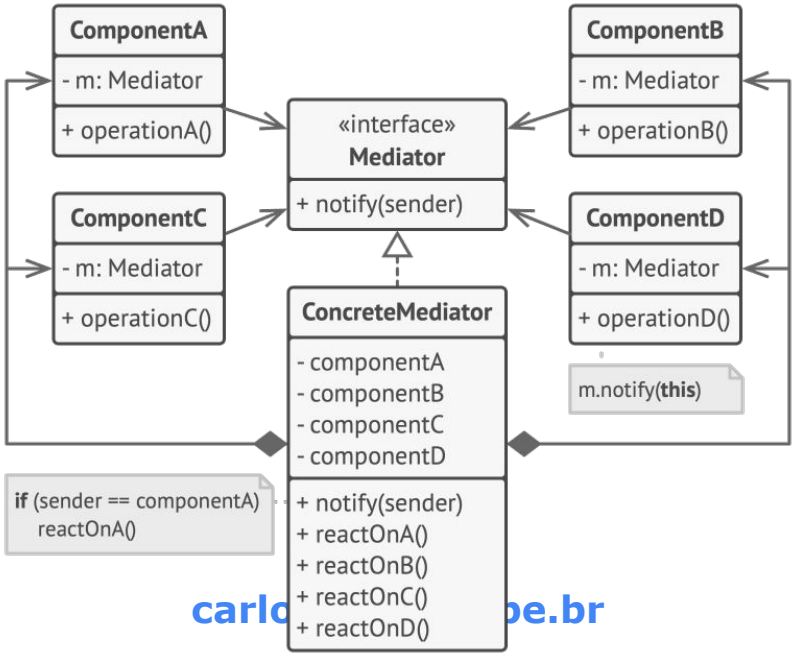
Mediator - Ideia e Intenção

"Encapsular as regras de colaboração entre objetos em um mediador"

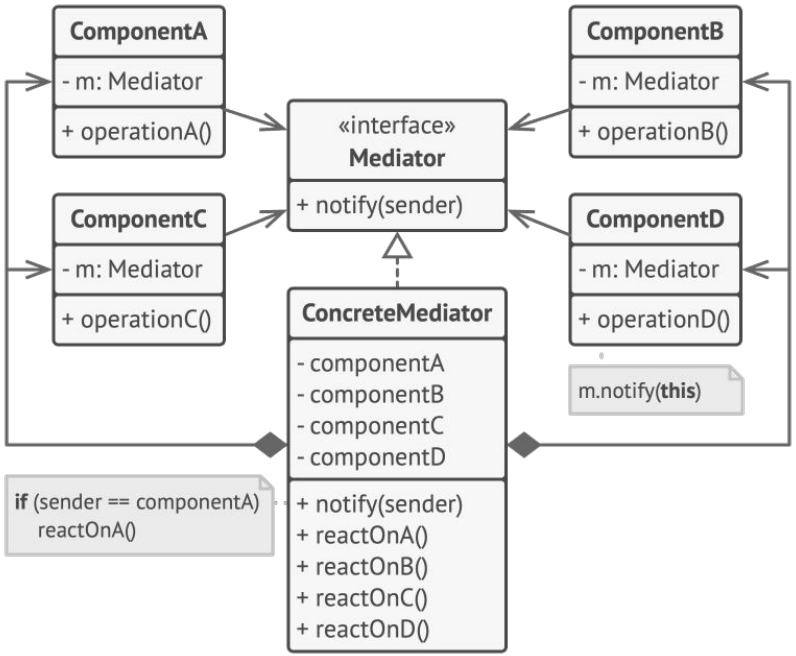
- No formulário:
  - A caixa de seleção informa que sua marcação mudou
  - O mediador atualiza a visibilidade do campo correspondente
  - O botão informa o clique
  - O mediador coordena a validação e o salvamento
- Usaremos:
  - Componentes: campos, caixa de seleção e botão
  - Mediator: interface de comunicação
  - ConcreteMediator: `FormularioPerfil`, que conhece e coordena os componentes

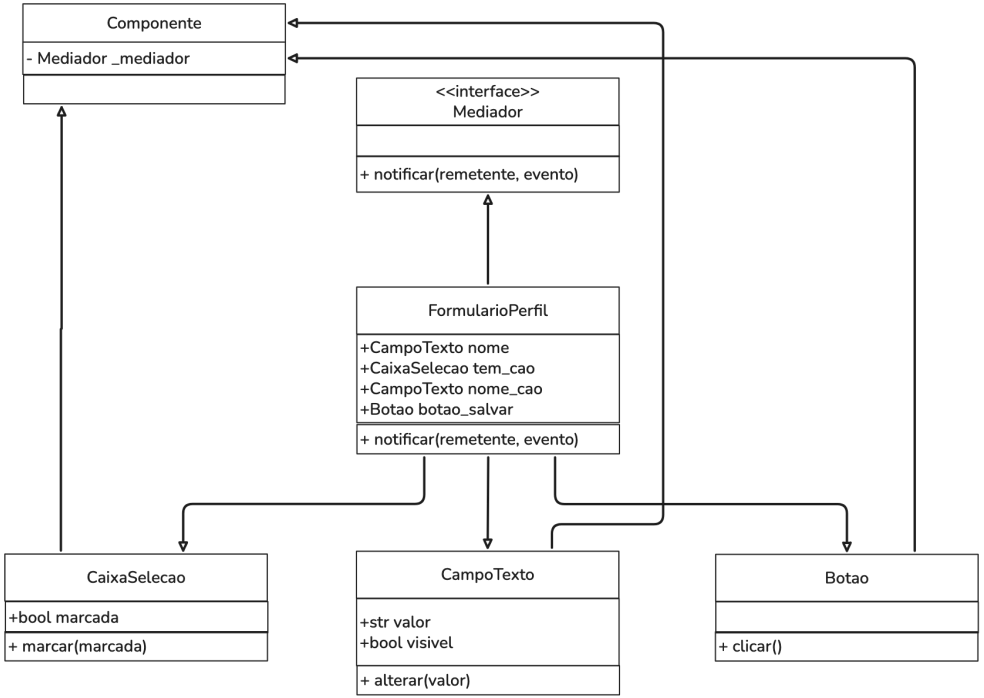
Mediator - Estrutura

1. Os **Components** mantêm uma referência à interface do mediador
2. O **Mediator** declara o contrato de comunicação
3. **ConcreteMediator** conhecem os componentes e implementam suas regras de colaboração
4. **Comunication** o componente notifica um evento e o mediator identifica o remetente e decide quais ações executar



Aplicando ao nosso contexto





Aplicando ao nosso contexto - contratos

```python
from abc import ABC, abstractmethod


class Mediador(ABC):
    @abstractmethod
    def notificar(self, remetente: "Componente", evento: str) -> None:
        pass


class Componente:
    def __init__(self, mediador: Mediador) -> None:
        self._mediador = mediador
```

- `Mediador` define a operação comum de comunicação
- `remetente` identifica o componente que originou o evento
- `evento` informa o que aconteceu
- `Componente` concentra a referência ao mediador, compartilhada pelas subclasses
- O contrato permite associar os componentes a diferentes implementações de mediador

Aplicando ao nosso contexto - campo de texto

```python
class CampoTexto(Componente):
    def __init__(self, mediador: Mediador, visivel: bool = True) -> None:
        super().__init__(mediador)
        self.valor = ""
        self.visivel = visivel

    def alterar(self, valor: str) -> None:
        self.valor = valor
        self._mediador.notificar(self, "alteracao")
```

- Cada campo mantém seu valor e sua visibilidade
- `alterar()` atualiza o valor antes de comunicar o evento
- O campo passa `self`, permitindo ao mediador identificar qual instância mudou
- A reação à alteração será definida pelo mediador

Aplicando ao nosso contexto - seleção e botão

```python
class CaixaSelecao(Componente):
    def __init__(self, mediador: Mediador) -> None:
        super().__init__(mediador)
        self.marcada = False

    def marcar(self, marcada: bool) -> None:
        self.marcada = marcada
        self._mediador.notificar(self, "marcacao")


class Botao(Componente):
    def clicar(self) -> None:
        self._mediador.notificar(self, "clique")
```

- A caixa de seleção atualiza sua marcação e avisa o mediador
- O botão informa que recebeu um clique
- A regra de exibir o campo do cão e a validação ao salvar ficarão no `FormularioPerfil`

Aplicando ao nosso contexto - mediador concreto

```python
class FormularioPerfil(Mediador):
    def __init__(self) -> None:
        self.nome = CampoTexto(self)
        self.tem_cao = CaixaSelecao(self)
        self.nome_cao = CampoTexto(self, visivel=False)
        self.botao_salvar = Botao(self)

    def notificar(self, remetente: Componente, evento: str) -> None:
        if remetente is self.tem_cao and evento == "marcacao":
            self.nome_cao.visivel = self.tem_cao.marcada

        elif remetente is self.botao_salvar and evento == "clique":
            self._salvar_perfil()
```

- O formulário cria os componentes e passa sua própria referência como mediador
- `remetente` identifica qual componente originou o evento
- A marcação controla a visibilidade do campo
- O clique no botão inicia a validação e o salvamento

Aplicando ao nosso contexto - validar e salvar

```python
    def _salvar_perfil(self) -> None:
        nome = self.nome.valor.strip()
        nome_cao = self.nome_cao.valor.strip()

        if not nome:
            print("Erro: informe o nome.")
            return

        if self.tem_cao.marcada and not nome_cao:
            print("Erro: informe o nome do cão.")
            return

        print(f"Perfil de {nome} salvo.")
```

- O nome do usuário é obrigatório
- O nome do cão é obrigatório quando a caixa está marcada
- `strip()` permite identificar valores preenchidos apenas com espaços
- O salvamento é simulado por uma mensagem
- As regras que envolvem diferentes componentes ficam concentradas no mediador

Aplicando ao nosso contexto - execução

```
formulario = FormularioPerfil()

formulario.botao_salvar.clicar()

formulario.nome.alterar("Ana")
formulario.tem_cao.marcar(True)
print("Campo do cão visível:", formulario.nome_cao.visivel)

formulario.botao_salvar.clicar()

formulario.nome_cao.alterar("Toby")
formulario.botao_salvar.clicar()

formulario.tem_cao.marcar(False)
print("Campo do cão visível:", formulario.nome_cao.visivel)

formulario.botao_salvar.clicar()
```

Prós e Contras do Mediator

Prós

- Concentra as regras de colaboração em um lugar
- Reduz as dependências diretas entre componentes
- Permite reutilizar componentes com outro mediador e outras regras

Contras

- O mediador pode crescer e concentrar responsabilidades demais
- Acrescenta uma camada de coordenação, cujo custo pode superar o benefício em interações simples

Use

- Quando os componentes possuem muitas dependências entre si, dificultando alterações
- Um componente é difícil de reutilizar porque contém regras específicas de colaboração com outros objetos
- Os mesmos componentes precisam participar de diferentes fluxos, coordenados por regras diferentes



Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br